

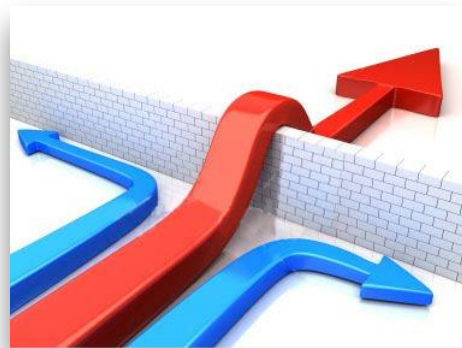
Aprendizado por Reforço

Aulas 11 e 12: Aprendizado por Diferença Temporal

Prof. Charles Madeira

Objetivo da aula

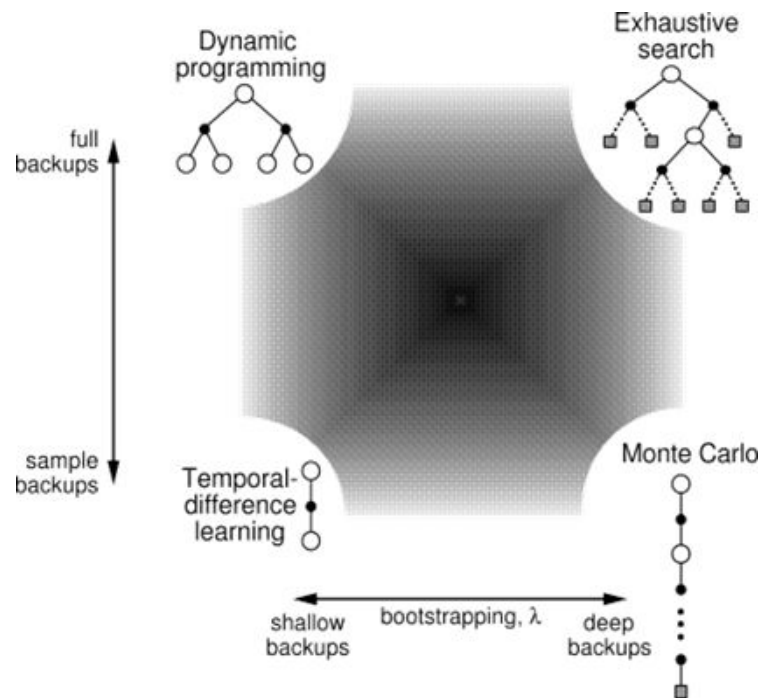
- Introduzir o aprendizado por Diferença Temporal (TD), que consiste em métodos fundamentais do aprendizado por reforço
- Apresentar os métodos de TD, que são voltados à predição e são capazes de aprender diretamente da experiência sem um modelo do ambiente
- Apresentar os métodos SARSA e Q-learning, que são voltados ao controle de TD visando aprender políticas ótimas



Aprendizado por Diferença Temporal

[Samuel 1959][Klopf 1972][Holland 1976, 1986][Sutton 1988]...

- Aprendizado por Diferença Temporal (TD *Learning*) é um conceito fundamental e inovador no aprendizado por reforço
- Atua como uma combinação de ideias dos métodos de Monte Carlo (MC) e da Programação Dinâmica (PD)
 - Recurso à experiência (amostragem) do MC, que pode aprender diretamente da experiência bruta, sem necessitar de modelo explícito de como o ambiente se comporta
 - Aspecto incremental (*bootstrapping*) da PD, que atualiza as estimativas com base em outras estimativas aprendidas, permitindo aprender de forma contínua, sem esperar o final do episódio

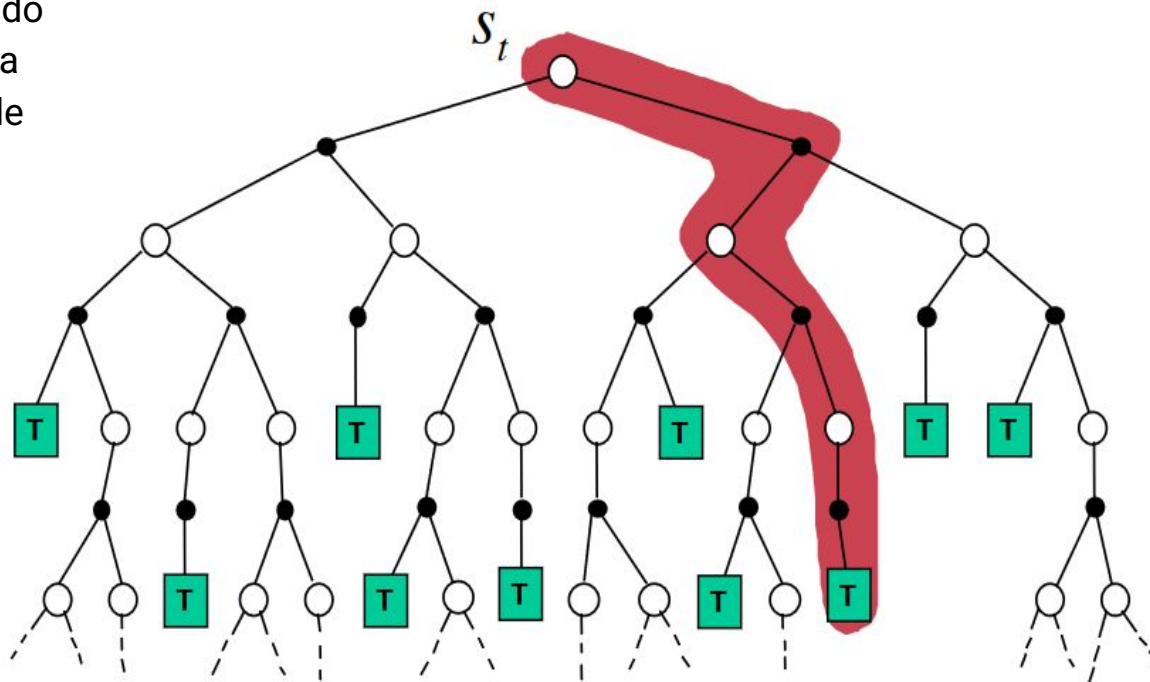


Backup do Monte Carlo

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^T \gamma^k r_{t+k+1}$$

Espera até o retorno completo G_t de um episódio ser conhecido para poder atualizar a estimativa do valor de um estado $V(S_t)$

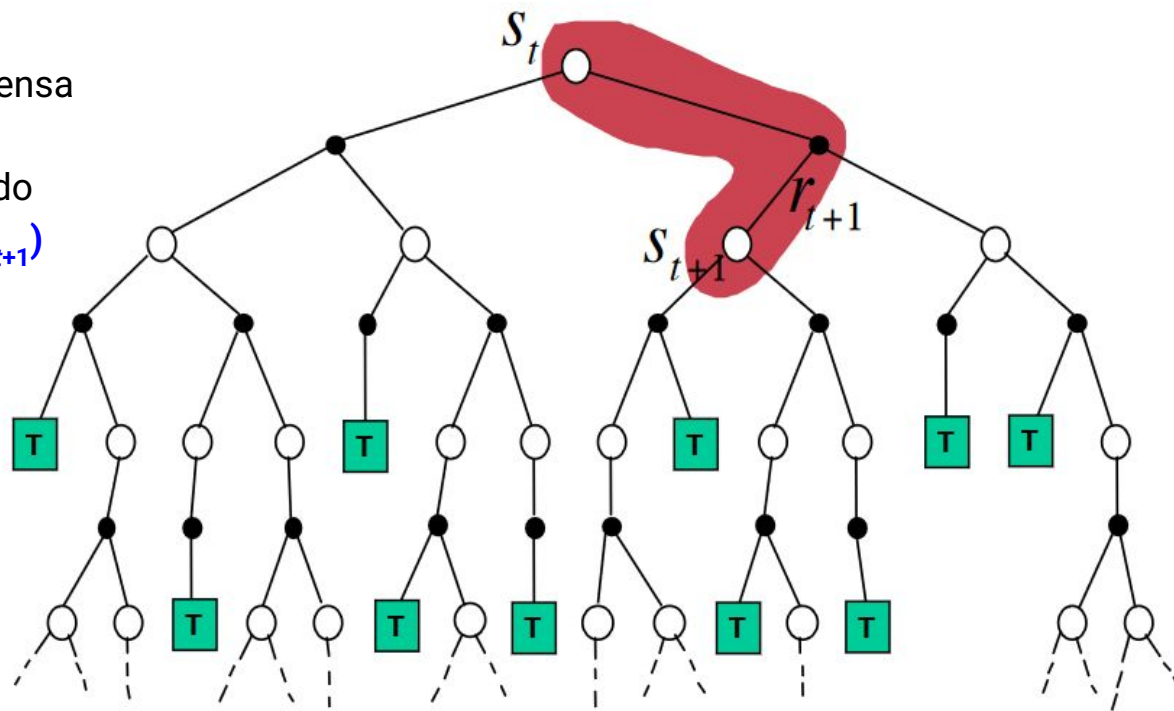
$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$



Backup da Diferença Temporal

Atualiza as estimativas com apenas um passo de tempo à frente, utilizando a recompensa observada R_{t+1} e a estimativa do valor do próximo estado $V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$



Método da Diferença Temporal (TD)

- TD é o método padrão do Aprendizado por Reforço, que consiste em comparar:
 - o retorno que o agente recebe efetivamente do ambiente referente a um estado s
 - o valor cumulativo do estado s presente na função de valor $V(s)$ (média dos retornos)
- Definição da equação de atualização incremental

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

$$V(s_t) \leftarrow V(s_t) + \alpha \delta_t$$

$V(s)$	: Função de valor dos estados
α	: Taxa de aprendizado
δ_t	: Erro da diferença temporal
r_{t+1}	: Recompensa imediata
γ	: Fator de desconto temporal

s	$V(s)$
s_0	R_t
s_1	$\dots$
$\dots$	$\dots$
s_n	$\dots$

- O erro de TD δ_t mede a diferença entre o valor estimado $V(s_t)$ e uma estimativa mais atualizada $R_{t+1} + \gamma V(s_{t+1})$ e torna-se disponível um passo de tempo após a estimativa ser feita

Predição de TD

- A Predição de Diferença Temporal utiliza a experiência para resolver o problema de predição
- **Atualização de estimativas**: ambos os métodos de TD e Monte Carlo atualizam suas estimativas V de V_π para os estados não terminais encontrados na experiência
- **TD(0)** é um método de *bootstrapping* porque baseia sua atualização em parte em uma estimativa existente, semelhante à PD
- Exemplo “Dirigindo para casa”:
 - Monte Carlo exige esperar até chegar em casa para aprender com o percurso total
 - TD permite o aprendizado imediato e contínuo, ajustando as estimativas à medida que novas informações (ex.: mudança no tráfego, chuva) surgem

Procedimento de aprendizado

- Exemplo do algoritmo TD(0), que trata apenas um passo

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

$A \leftarrow$ action given by π for S

 Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

 until S is terminal



Vantagens dos Métodos de Predição de TD

- **Não requer modelo do ambiente:** métodos de TD têm vantagem sobre os métodos DP, pois não exigem um modelo do ambiente (função de probabilidade de transição T), sendo caracterizados com *free model*
- **Implementação online e incremental:** métodos TD têm vantagem sobre os métodos MC, pois são naturalmente implementados de forma online, totalmente incremental
 - MC exige esperar o fim de um episódio para o retorno ser conhecido
 - TD precisa esperar apenas um passo de tempo, o que é crucial para aplicações com episódios muito longos ou tarefas contínuas sem episódios
- **Convergência:** TD(0) converge para $V\pi$
 - na média para um tamanho de passo constante e suficientemente pequeno
 - com probabilidade 1 se o tamanho de passo diminuir de acordo com as condições usuais de aproximação estocástica
- **Eficiência:** empiricamente, os métodos de TD geralmente convergem mais rapidamente que os métodos de MC em tarefas estocásticas, em termos de **erro quadrático médio** (MSE)

Otimização do TD(0)

- Quando há uma quantidade finita de experiência e é apresentada repetidamente (atualização em batch):
 - **Convergência**: sob atualização em *batch*, TD(0) e MC convergem deterministicamente, mas para resultados diferentes
 - **Monte Carlo (*batch*)**: converge para valores que são médias amostrais dos retornos reais experimentados, sendo ótimos no sentido de minimizar o erro quadrático médio dos retornos no conjunto de treinamento
 - **TD(0) (*batch*)**: converge para as estimativas que seriam exatamente corretas para o modelo de máxima verossimilhança do processo de Markov, que é chamado de **estimativa de equivalência de certeza**
 - **Rapidez**: a relação com a estimativa de equivalência de certeza pode explicar a razão dos métodos de TD convergirem mais rapidamente, embora não seja computacionalmente viável calcular essa estimativa para grandes espaços de estados

SARSA: Controle TD On-policy $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$

- SARSA é um método de **controle TD on-policy**, que avalia e melhora a mesma política que está sendo usada para gerar as ações para aprender uma função
- Se assemelha ao TD, tendo a função de valor dos estados $V(s)$ substituída pela função de valor das ações $Q(s,a)$
- Definição da equação de atualização incremental

$$\delta_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \delta_t$$

$Q(s,a)$	: Função de valor das ações
α	: Taxa de aprendizado
δ_t	: Erro da diferença temporal
r_{t+1}	: Recompensa imediata
γ	: Fator de desconto temporal

s	a	Q(s,a)
s_0	a_0	R_t
s_0	a_1	...
...	...	...
s_n	a_0	...
s_n	a_1	...

- O fato de sempre determinar no instante t a ação que será utilizada no instante $t+1$ significa que o agente segue a estratégia adotada (política) -> *on-policy*

SARSA: Controle TD On-policy

- SARSA estima a utilidade de uma ação a partir de um estado (S, A) ao invés da utilidade de estar em um determinado estado (S)
- **Transições de pares estado-ação**: as transições são consideradas de um par estado-ação para outro, usando a quintupla de eventos ($S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$), que dá nome ao método
- **Convergência**: SARSA converge com probabilidade 1 para uma política e função de valor Q^* , desde que todos os pares estado-ação sejam visitados infinitas vezes e a política convirja para uma política gananciosa

Procedimento de aprendizado

- Exemplo do algoritmo SARSA

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

Q-learning: Controle TD Off-policy [Watkins 1989]

- Q-learning é uma simplificação do SARSA pelo fato que não é mais necessário determinar, no instante t , a ação que será utilizada no instante $t+1$
- Definição da equação de atualização incremental

$$\delta_t = r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \delta_t$$

$Q(s,a)$: Função de valor das ações
 α : Taxa de aprendizado
 δ_t : Erro da diferença temporal
 r_{t+1} : Recompensa imediata
 γ : Fator de desconto temporal

s	a	$Q(s,a)$
s_0	a_0	R_t
s_0	a_1	...
...	...	...
s_n	a_0	...
s_n	a_1	...

- Ao invés de determinar a ação antecipadamente, o agente escolhe aquela ação que tem o melhor valor estimado
 - O agente não segue uma estratégia (política) -> *off-policy*

Q-learning: Controle TD Off-policy [Watkins 1989]

- Q-learning foi um dos primeiros avanços no aprendizado por reforço e é um algoritmo de **controle TD off-policy**
- A função de valor de ação Q aprendida aproxima a função de valor de ação ótima Q^* , independente da política que está sendo seguida
- A principal diferença em relação ao SARSA é o uso de $\max_a Q(S_{t+1}, a)$ como o alvo, o que significa assumir que a próxima ação será a ótima, independentemente da política de comportamento atual
- **Convergência**: converge com probabilidade 1 para Q^* sob a suposição de que todos os pares estado-ação continuam sendo atualizados
- É algoritmo de Aprendizado por Reforço mais conhecido em razão de haver provas formais de convergência [Watkins e Dayan 1992]

Procedimento de aprendizado

- Exemplo do algoritmo Q-learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

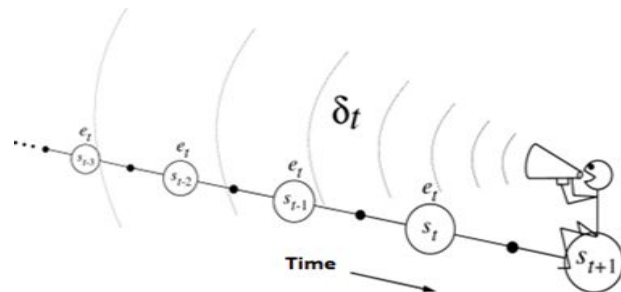
 until S is terminal

SARSA vs Q-learning

- Ambos algoritmos funcionam mantendo uma tabela de valores Q para cada par estado-ação
 - A cada intervalo de tempo, o agente seleciona uma ação com base no estado atual e executa a ação no ambiente
 - O agente observa a recompensa e o próximo estado e atualiza os valores Q para um par estado-ação com base no reforço e no valor Q estimado do próximo par estado-ação
- SARSA é um algoritmo *on-policy*, que atualiza os valores Q com base nas ações que são realmente executadas no ambiente, em vez de ter base nas ações que seriam executadas de acordo com a política ideal
 - Isso o torna mais estável, visando convergir para a política ideal em comparação com algoritmos *off-policy*, como o Q-learning
 - No entanto, pode ser mais lento para convergir em comparação com algoritmos *off-policy*, especialmente em ambientes com recompensas retardadas
 - Também é mais sensível à taxa de aprendizado, que determina o quanto os valores Q são atualizados a cada intervalo de tempo

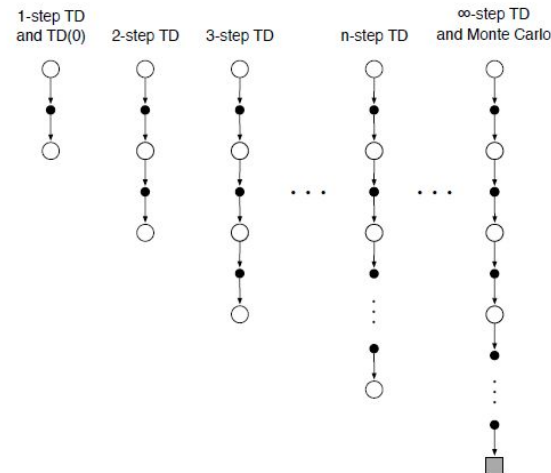
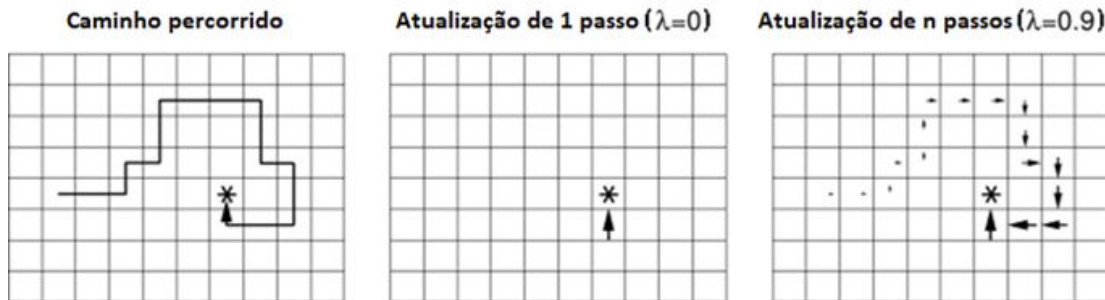
TD(λ), SARSA (λ) e Q(λ)

- Um possível problema dos algoritmos precedentes é que eles somente atualizam um valor a cada instante
 - O procedimento de atualização pode ser tornar particularmente lento
- Portanto, é possível dotá-los de uma memória de transição (histórico)
 - Cada estado é associado a um indicador $e(s)$ que mede a evolução do tempo desde a última vez que ele foi visitado
 - O indicador $e(s)$:
 - recebe um valor igual a 1 para o estado que está sendo visitado
 - diminui de um fator $\gamma\lambda$ para os outros estados
 - O erro da diferença temporal corrige simultaneamente a estimativa do valor dos estados em função do valor de $e(s)$



TD(λ), SARSA (λ) e Q(λ)

- O fator λ introduz um mecanismo de bootstrapping
 - Se $\lambda=0$, voltamos ao TD(0), SARSA(0) e Q-learning(0)
 - Se $\lambda=1$, os algoritmos se aproximam do Monte Carlo



- Os algoritmos dotados de histórico podem ser mostrar mais eficazes que suas versões padrão, mas podem requerer bastante espaço de memória
 - É necessário encontrar um bom compromisso entre desempenho do treinamento e uso do espaço de memória

Procedimento de aprendizado

- Exemplo do algoritmo TD(λ)

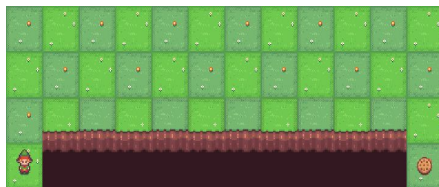
```
Initialize  $V(s)$  arbitrarily and  $e(s) = 0$ , for all  $s \in \mathcal{S}$ 
Repeat (for each episode):
  Initialize  $s$ 
  Repeat (for each step of episode):
     $a \leftarrow$  action given by  $\pi$  for  $s$ 
    Take action  $a$ , observe reward,  $r$ , and next state,  $s'$ 
     $\delta \leftarrow r + \gamma V(s') - V(s)$ 
     $e(s) \leftarrow e(s) + 1$ 
    For all  $s$ :
       $V(s) \leftarrow V(s) + \alpha \delta e(s)$ 
       $e(s) \leftarrow \gamma \lambda e(s)$ 
     $s \leftarrow s'$ 
  until  $s$  is terminal
```

Exercícios

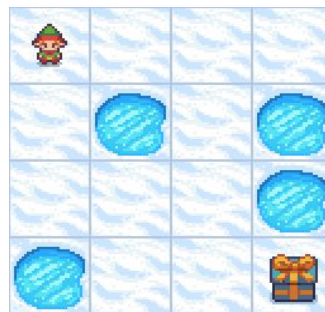
- Experimentar os algoritmos Q-learning(0) e SARSA(0) nos ambientes [Toy Text do Gymnasium](#)



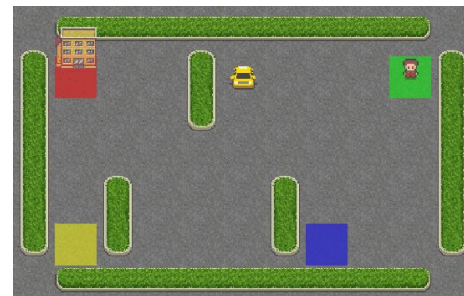
Blackjack



Cliff Walking



Frozen Lake



Taxi

- Na pasta do Drive, tem exemplos do [Q-learning](#) e do [SARSA](#) para o Blackjack (inspirado no [tutorial do Gymnasium](#))